1. Compare the triplets:

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>

int main(){
   int a0;
   int a1;
   int a2;
   scanf("%d %d %d",&a0,&a1,&a2);
   int b0;
   int b1;
   int b2;
  int a_score = 0;
   int b_score = 0;
   scanf("%d %d %d",&b0,&b1,&b2);
   if (a0 > b0)
      a_score++;
   else if (a0 < b0)
      b_score++;
   else{}
      //no op
   if (a1 > b1)
      a_score++;
   else if (a1 < b1)
      b_score++;
   else {}
```

```
    //no op
  if (a2 > b2)
    a_score++;
  else if (a2 < b2)
    b_score++;
  else {}
    //no op
   printf("%d %d",a_score, b_score);
   return 0;
}
```

2. Diagonal Difference

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main()
{
  int n, j;
  int i=0,RightDiagonalSum=0,LeftDiagonalSum=0, firstarray, secondarray;
 scanf("%d",&n);
 int a[n][n];
for( firstarray = 0; firstarray < n; firstarray++)
{
    for( secondarray = 0; secondarray < n; secondarray++)
  {
          scanf("%d",&a[firstarray][secondarray]);
    }
  }
```

```
  while(i<n)
 {
   RightDiagonalSum=RightDiagonalSum+a[i][i];
   i++;
 }
   j=n-1,i=0;
   while(i<n)
 {
   LeftDiagonalSum=LeftDiagonalSum+a[i][j];
   i++;
   j--;
 }
  printf("%d",abs(RightDiagonalSum-LeftDiagonalSum));
 return 0;
}
```

3. Non-Divisible Subset

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Helper functions for min and max
int min(int a, int b) {
   return a < b ? a : b;
}
int max(int a, int b) {
   return a > b ? a : b;
}

int main() {
   int n, k, a, total = 0;
   // Read input n and k
   scanf("%d %d", &n, &k);
```

```
    // Array to store counts of each remainder from 0 to k-1
    int mods[k];
    for (int i = 0; i < k; i++) {
        mods[i] = 0;
    }
    // Read all numbers and count their remainders
    for (int i = 0; i < n; i++) {
        scanf("%d", &a);
        mods[a % k]++;
    }
    // Add at most one element with remainder 0
    total += min(1, mods[0]);
    // Handle complementary remainders
    for (int d = 1; d < (k + 1) / 2; d++) {
        // Pick the maximum count between remainder d and remainder k-d
        total += max(mods[d], mods[k - d]);
    }
    // If k is even, add at most one element with remainder k/2
    if (k % 2 == 0) {
        total += min(1, mods[k / 2]);
    }
    // Print the size of the largest non-divisible subset
    printf("%d\n", total);
    return 0;
}
```

4. **Bear and Steady Gene**

```
#include <stdio.h>
#include <string.h>
#include <math.h>
```

```
#include <stdlib.h>
// Function to check if all characters outside the window are balanced (<= n/4)
int is_balanced(int count[], int target) {
    if (count['A'] > target || count['C'] > target || count['G'] > target || count['T'] > target) {
        return 0; // Not balanced
    }
    return 1; // Balanced
}
int steadyGene(char* gene, int n) {
    int target = n / 4;
    int count[128] = {0}; // Using ASCII values as indices for counts
    // Initial pass: Count all characters
    for (int i = 0; i < n; i++) {
        count[gene[i]]++;
    }
    // If already steady, the required substring length is 0
    if (is_balanced(count, target)) {
        return 0;
    }
    int min_len = n;
    int left = 0;
    // Sliding window: right pointer expands the window
    for (int right = 0; right < n; right++) {
        // Decrease count of the character entering the window (remaining outside the balanced region)
        count[gene[right]]--;
        // Inner while loop: left pointer shrinks the window from the left
        // as long as the remaining characters outside the window are balanced
        while (is_balanced(count, target)) {
            // A valid window is found, update the minimum length
            // The length of the substring to be replaced is (right - left + 1)
```

```
            if ((right - left + 1) < min_len) {
                min_len = (right - left + 1);
            }
            // Shrink the window from the left: increase count of the character leaving the window
            count[gene[left]]++;
            left++;
        }
    }
    return min_len;
}
int main() {
    int n;
    // Read the gene length
    if (scanf("%d", &n) != 1) return 1;
    char* gene = (char*)malloc((n + 1) * sizeof(char));
    // Read the gene string
    if (scanf("%s", gene) != 1) {
        free(gene);
        return 1;
    }
    int result = steadyGene(gene, n);
    printf("%d\n", result);
    free(gene);
    return 0;
}
```